

Taller de Lógica: Comunicación mediante Tuberías

Malak Sanchez

Monitorías de Sistemas Operativos

16 de abril de 2026

Introducción

Este taller se enfoca en el uso de tuberías anónimas (`pipe`) para el flujo de datos entre procesos. Se explorarán conceptos como la redirección de descriptores de archivos (`dup2`), la ejecución de comandos externos (`exec`) y la comunicación bidireccional.

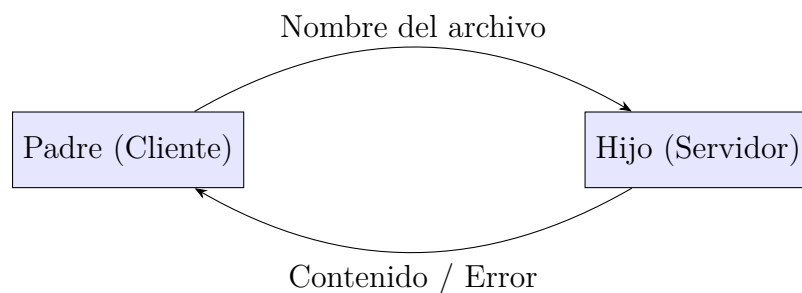
Bloque 1: Implementación de Tuberías Básicas

Utilice `fork()` y `pipe()` para implementar flujos de datos entre procesos y redireccionar la salida estándar si es necesario.

Ejercicio 1: Lectura Remota de Archivos

Este ejercicio simula un servidor de archivos básico. El proceso padre actúa como cliente y el hijo como servidor.

1. El Padre envía el nombre de un archivo (ej: `"datos.txt"`) al hijo a través de una tubería.
2. El Hijo abre el archivo, lee su contenido y lo envía de vuelta al padre mediante una segunda tubería.
3. El Padre imprime en pantalla el contenido recibido.
4. **Reto:** El hijo debe enviar un mensaje de error si el archivo no existe.



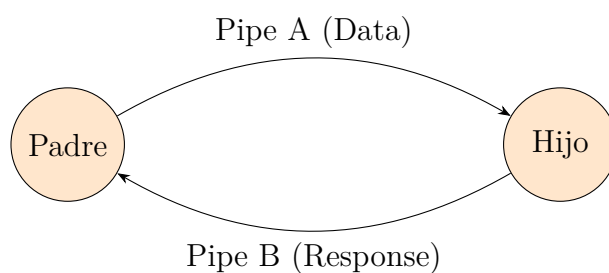
Bloque 2: Comunicación de Datos Dinámicos

Desarrolle programas que intercambien información estructurada entre procesos.

Ejercicio 2: El Eco Bidireccional

Implemente una comunicación de ida y vuelta.^{en}tre un padre y su hijo utilizando **dos tuberías**.

1. El Padre lee una cadena por teclado y la envía por el **Pipe A**.
2. El Hijo recibe la cadena, la convierte a MAYÚSCULAS y la envía por el **Pipe B**.
3. El Padre recibe la respuesta y la imprime en pantalla.



Ejercicio 3: El Recolector de Estadísticas

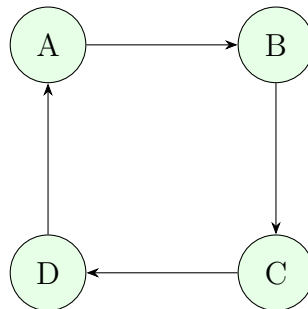
Un proceso padre crea N procesos hijos. Cada hijo debe:

1. Generar un número aleatorio entre 1 y 100.
2. Enviar dicho número al padre a través de una tubería **compartida** o una tubería por cada hijo.

El padre debe esperar a que todos los hijos terminen, recibir los números y calcular el promedio, el máximo y el mínimo.

Ejercicio 4: El Anillo de Procesos

Cree una estructura de anillo con 4 procesos ($A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$).



Un "token" numérico debe viajar por el anillo. Cada proceso debe sumarle 1 al token y pasarlo al siguiente. El juego termina cuando el token da 5 vueltas completas al anillo.

Bloque 4: Análisis de Código

Analice el fragmento y responda: ¿Por qué el programa se queda colgado (deadlock) y no termina?

```
1  int fd[2];
2  pipe(fd);
3  if (fork() == 0) {
4      char buf[10];
5      // El hijo no cierra nada
6      read(fd[0], buf, 10);
7      printf("Hijo leyo: %s\n", buf);
8      exit(0);
9  }
10 // El padre tampoco cierra fd[1] explícitamente antes de esperar
11 write(fd[1], "Hola", 5);
12 wait(NULL);
```